

Introduction to Computer Architecture

Chapter 5

Large and Fast: Exploiting Memory Hierarchy - 2

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

Improving Cache Performance

① Associativity

❖ Drawback of a direct-mapped cache:

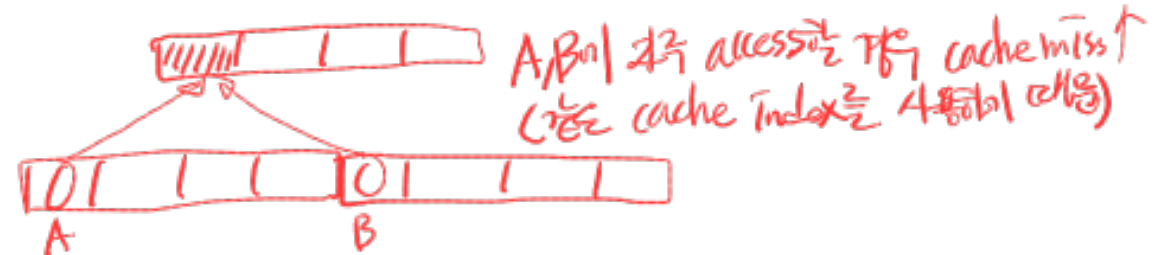
➤ If frequently used addresses are mapped to the same cache index, many cache misses will happen!

→ Solution: Multiple slots per index!

simplicity: cache를 access할 때 2개씩만
위치를 확인하면 될

Larger memory cache miss↓,

but access time↑
그래서 cache memory size는 한계가 있을 거고
→ cache hit↑, cache miss↓ 가능할 것



② Multi-level Cache

❖ Small cache has high miss rate

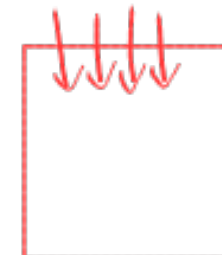
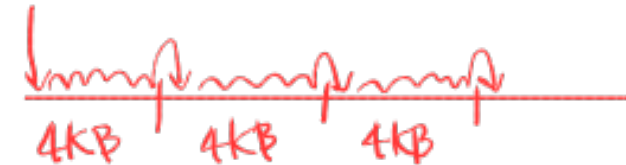
❖ Large cache has slow hit time

→ Solution: Use multiple levels of caches!



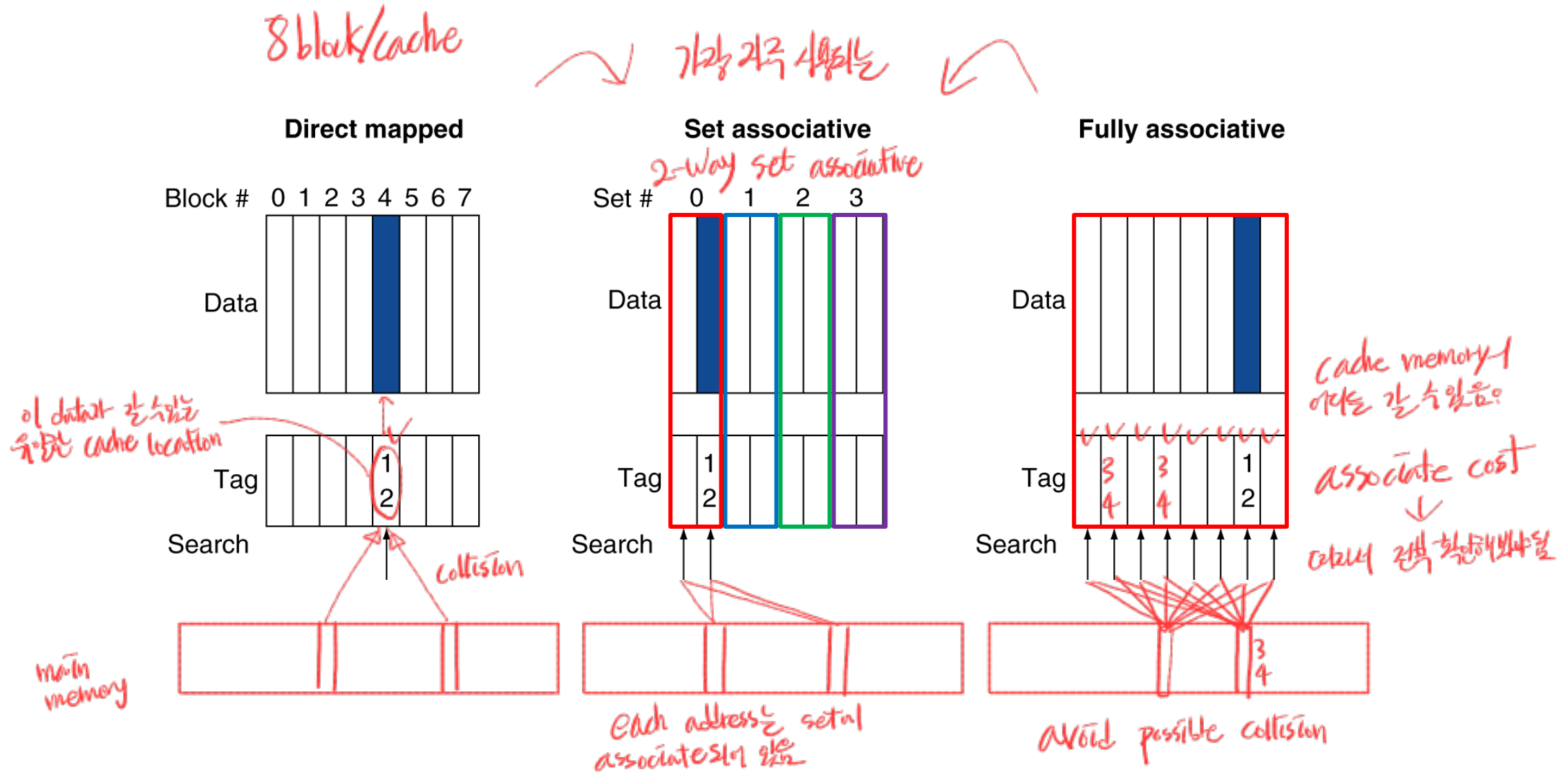
Software Optimization

❖ Solution: Strategically design data access pattern



random으로
memory에 2개씩
접근 access
cache를 활용하기 위해
access order를 rearrange

Associative Cache Example



Associative Caches

■ Fully associative

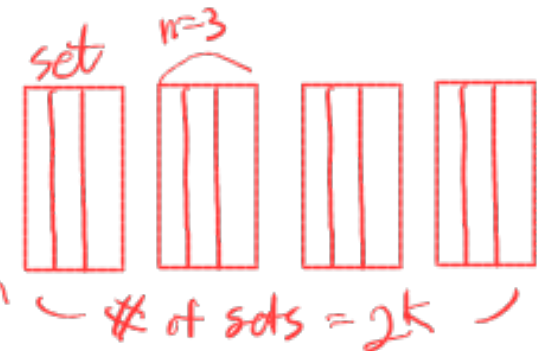
- ❖ Allow a given block to be placed anywhere in the cache memory.
- ❖ Requires all locations to be searched at once (Otherwise, very slow..)
- ❖ A comparator for each entry (expensive)

■ n -way set associative

- ❖ Each **set** contains n locations (n blocks)
- ❖ Block number determines the set index
 - (Block number) modulo (#Sets in cache)
- ❖ Search all entries in a given set at once
- ❖ n comparators (less expensive)
- ❖ n may not be a power-of-two

*Block # mod 4
(index 2, set 2, 12, 31, 35)*

**Block → have to 2^K
n-way set associative $\frac{2^K}{n}$ X
3-way도 가능 ($\frac{2^K}{3}$)*



Spectrum of Associativity

- For a cache with 8 block locations

**One-way set associative
(direct mapped)**

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

2 blocks
← single set

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Cache Index X

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Associativity Example

- Compare 4-block caches
 - ❖ Direct mapped / 2-way set associative / fully associative
 - ❖ Block address access sequence: 0, 8, 0, 6, 8

cache index = block address mod 4

- Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[6]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

all miss

*0, 8
4th conflict*

Associativity Example

- Fully associative → 모든 주소가 어디든 갈수있는 (only one large set) ~~구분X~~

Block address		Hit/miss	Cache content after access			
0		miss	Mem[0]			
8		miss	Mem[0]	Mem[8]		
0		hit	Mem[0]	Mem[8]		
6		miss	Mem[0]	Mem[8]	Mem[6]	
8		hit	Mem[0]	Mem[8]	Mem[6]	

Associativity Example

■ 2-way set associative

0 8 0 6 8 mod 2

↓ ↓ ↓ ↓ ↓ set이 2block

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	Mem[0]			
8	0	miss	Mem[0]	Mem[8]		
0	0	hit	Mem[0]	Mem[8]		
6	0	miss	Mem[0]	Mem[8]		
8	0	miss	Mem[8]	Mem[6]		

0, 8, 6이 같은 set에
들어감 competition이 존재

Mem[8]이 아니라 Mem[6]을
replace 할까? 아니면 hit로 바꿀까?
무엇을 선택하느냐에 따라 cache performance가 달라질

How Much Associativity

- Increased associativity generally decreases miss rate

- ❖ But with diminishing returns

higher associativity
→ lower miss rate obtain
lower set of set competition
→ hit rate ↑ miss rate ↓
higher hit rate
lower miss rate

- Simulation of a system with 64KB
D-cache, 16-word blocks, SPEC2000

- ❖ 1-way: 10.3%

- ❖ 2-way: 8.6%

- ❖ 4-way: 8.3%

- ❖ 8-way: 8.1%

increasing associativity
miss rate generally decrease

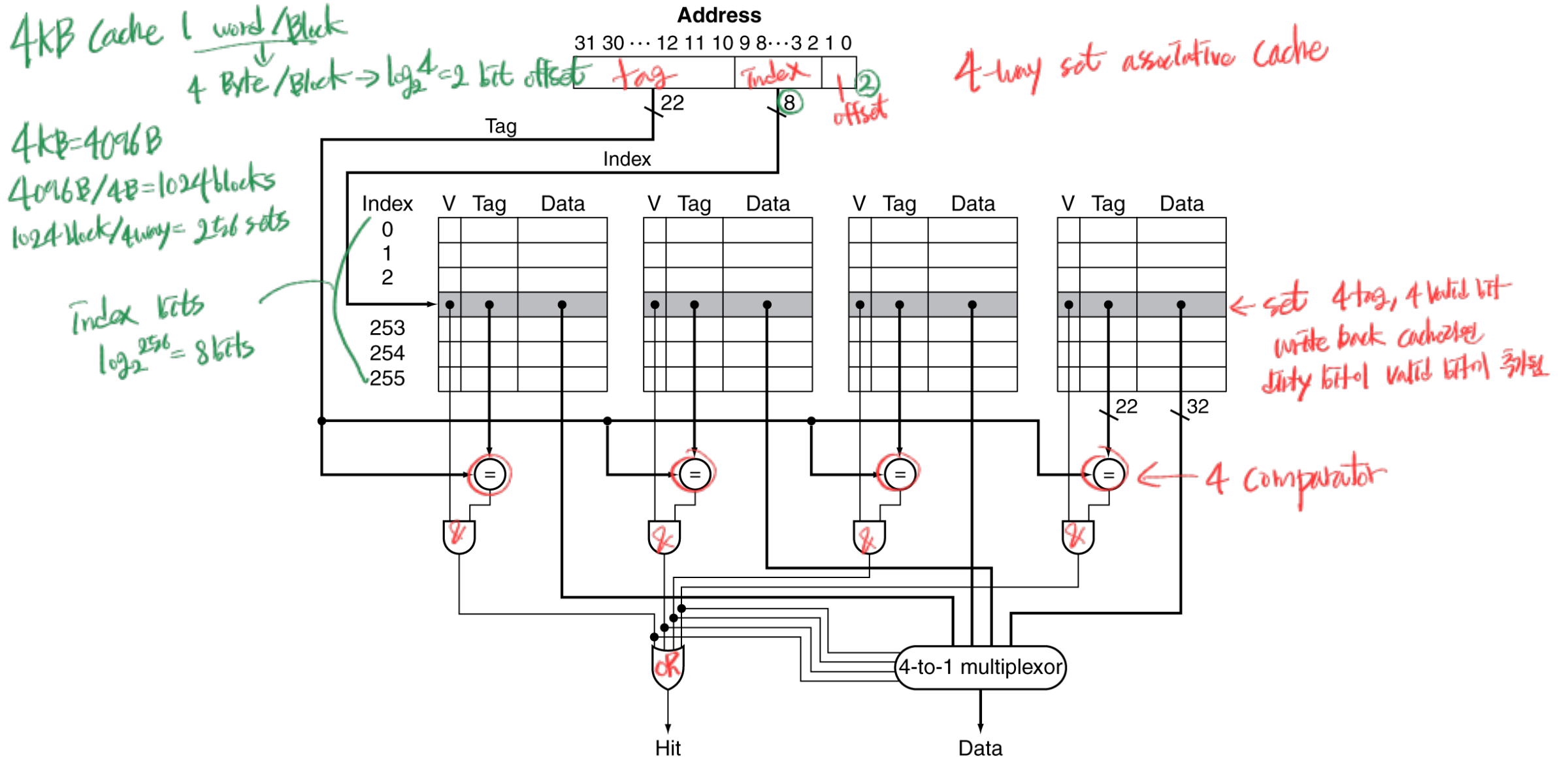
↓
효과가 줄어들고
여유공간이 남는 것

L1 cache

4, 8, 16 - way

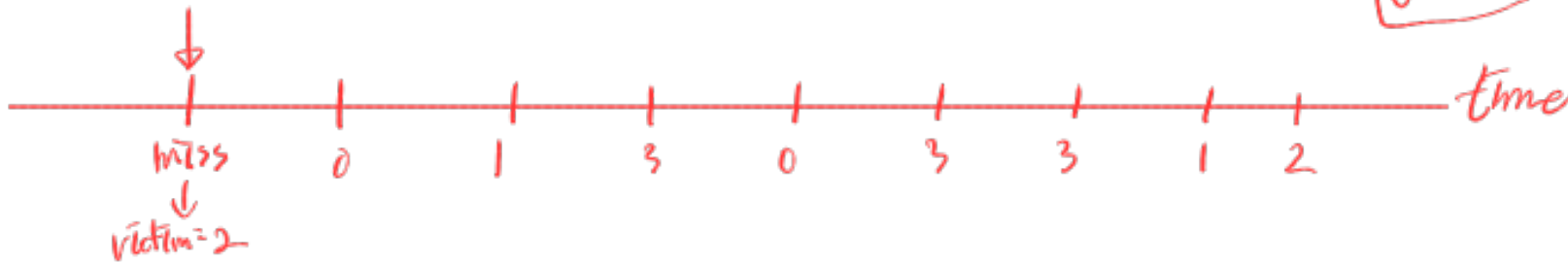
4 ~ 16 way CPU에서 4-way associativity

Set Associative Cache Organization



Replacement Policy

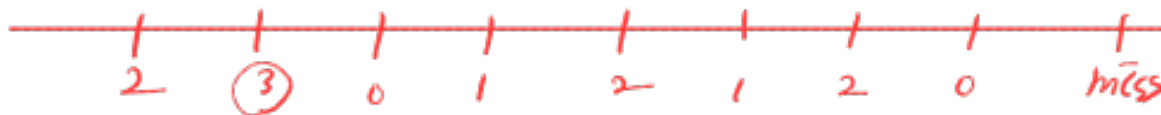
- Direct mapped: no choice (이런 2개강)
- Set associative
 - ❖ Choose a non-valid (empty) location, if there is any.
 - ❖ Otherwise, need to choose one of the blocks in the set as the **victim**
- The best choice? The block that will be used in the most distant future. *Best는 미래를 예측하는 것*
 - ❖ Theoretically true, but impossible to know



Replacement Policies for Set Associative 1

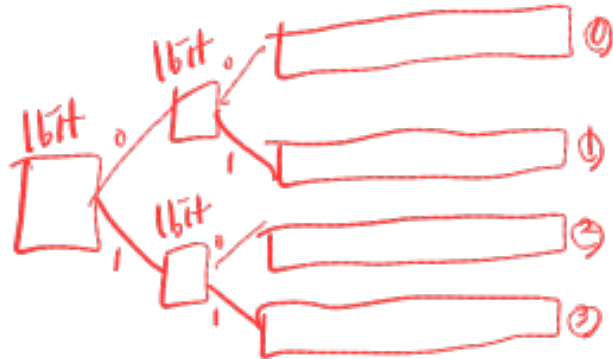
■ Least-recently used (LRU)

- ❖ Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that



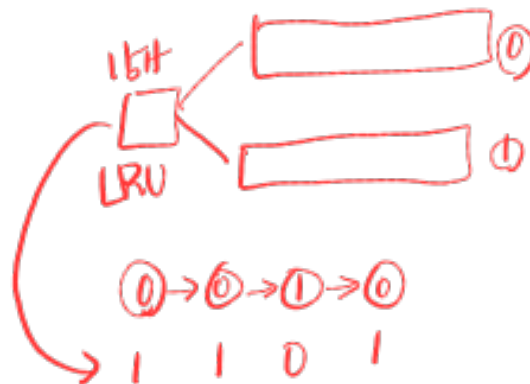
■ Pseudo LRU (for 4+ ways)

- ❖ Use an approximation
- ❖ e.g., A tree-like structure that has an LRU bit per 2 entries.



이런 data를
24bit로 만들거야

0 → 3 → 1
1 1 0 1 0 1 0



BestX
But cost면 LRU보다 좋은

4! = 24
↓
5bit
3bit

4 2 5
↓ ↓ ↓
0 3 2 1 4
↑ 3 1 4 2
↑ 1 4 2 5
↑
obst

n element
all permutative
n! 4 element
↓
24개 가능
↓
비교를 증가
하드웨어가 커질수록
Cache miss와 상관없이
비트 update
2 way로 바뀌는
상당 부분의 경우
Best는 아니

가장 오래 사용 안한 3이 victim
→ 4중에서도 사용 안한 거라고 예측

Replacement Policies for Set Associative 2

■ Sequential (Round Robin)

- ❖ Select the replacement victim in the circular order (i.e., $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \rightarrow 1 \dots$)

■ Random

- ❖ Similar performance as LRU if the associativity is high enough.



$n \uparrow$
이 경우
random, sequential한
순서로 접근할 때
3, 4, 2를 victims로 선정할 수 있음
따라서 유리할 수 있음

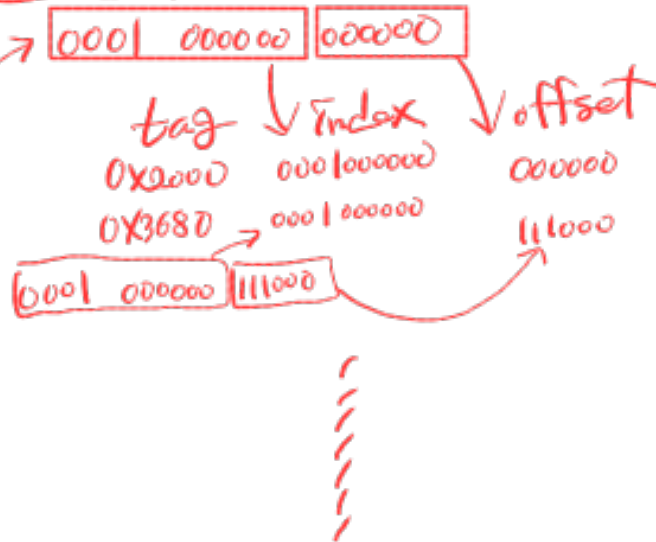
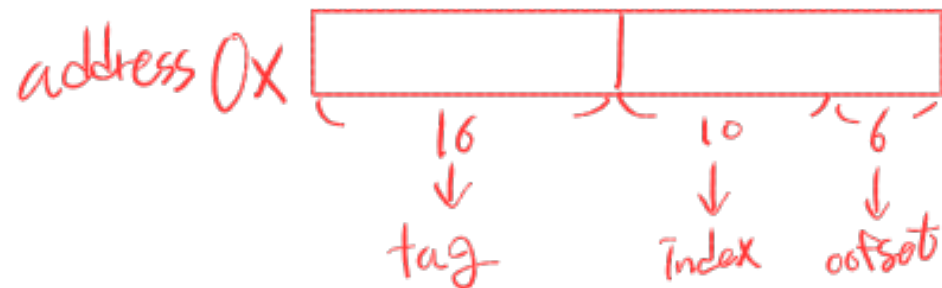
이 경우
pseudo LRU와
heuristic LRU가
가능

$$n=8) \quad \frac{3}{8} \quad n=32) \quad \frac{3}{32} \downarrow$$

Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

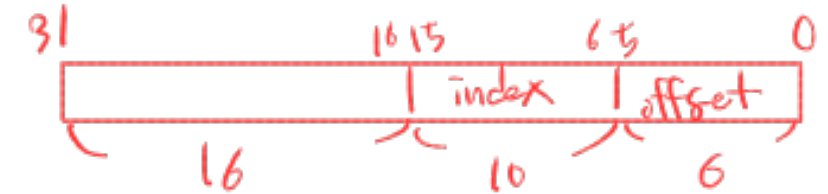
- Load 0x20001000
- Store 0x36801038
- Load 0x20001080
- Load 0x20001008
- Load 0x40811024
- Load 0x53841010
- Store 0x53841020
- Load 0x20001008



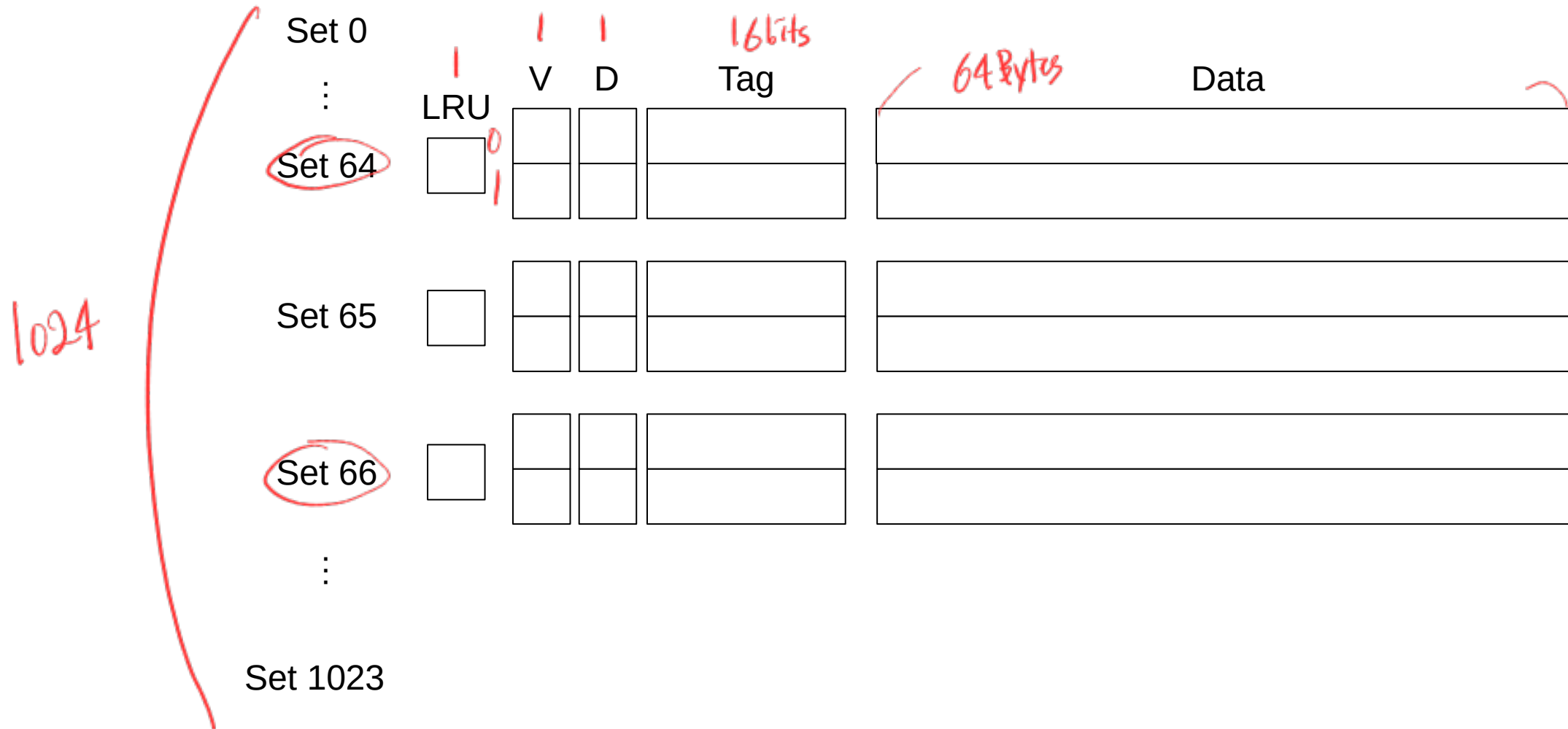
$$2^7 \quad 2^{10} \quad 2^6$$

$$128 \times 1024 \text{ Bytes} / 64 \text{ Bytes} = 2048 \text{ Blocks}$$

$$2048 / 2 = 1024 \text{ sets} \quad \log_2 1024 = 10$$



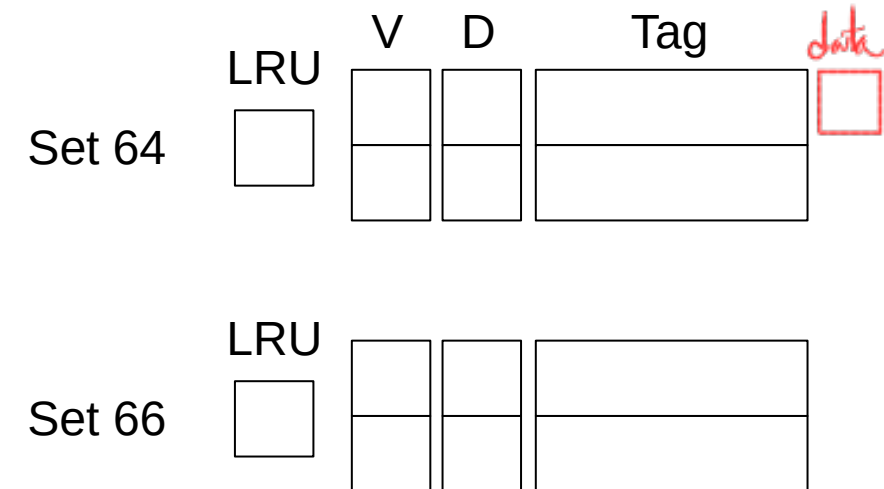
Cache Example



Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

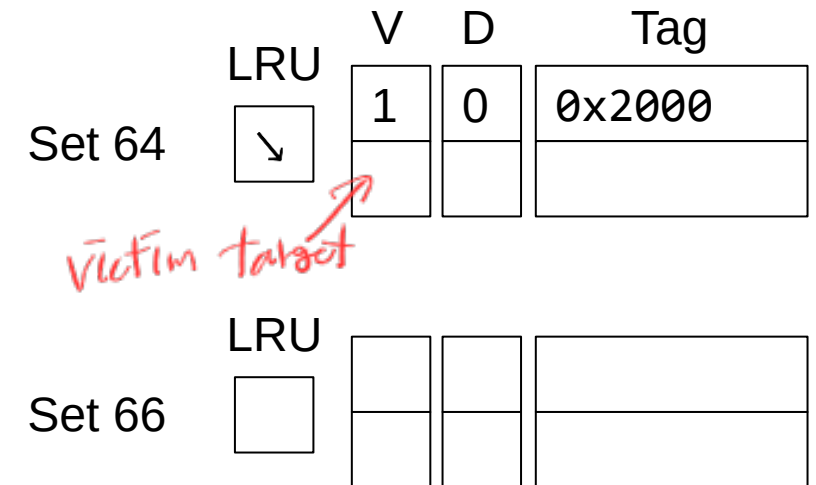
■ Load address tag index offset
 0x20001000 0x2000 0001000000₂ → 64₁₀ 000000₂



Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

	address	tag	index	offset	
■ Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss



Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

	address	tag	index	offset	
■ Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
■ Store	0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss

	LRU	V	D	Tag
Set 64		1	0	0x2000
		1	1	0x3680
Set 66				

Store operation,
write back to
dirty bit set

Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

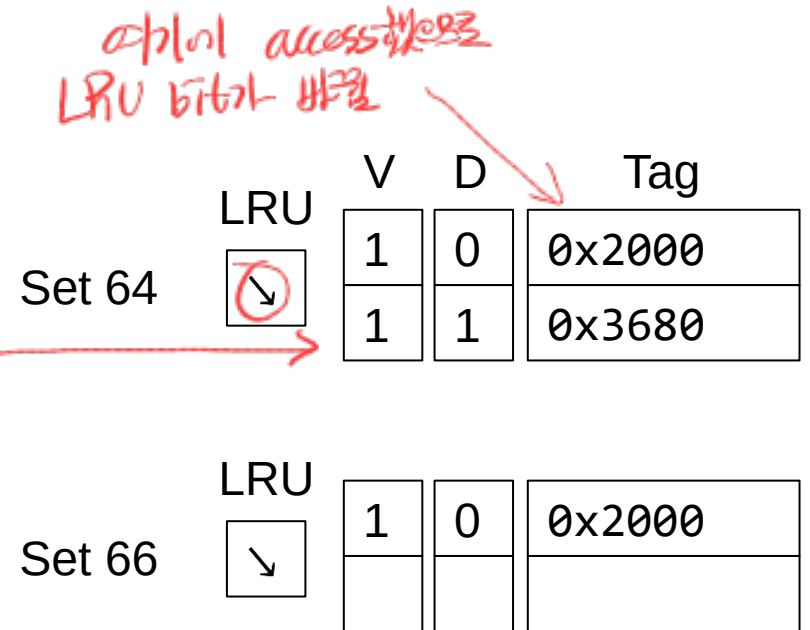
	address	tag	index	offset	
■ Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
■ Store	0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss
■ Load	0x20001080	0x2000	0001000010 ₂ →66 ₁₀	000000 ₂	miss

	LRU	V	D	Tag
Set 64	↗	1	0	0x2000
		1	1	0x3680
Set 66	↘	1	0	0x2000

Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

	address	tag	index	offset	
■ Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
■ Store	0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss
■ Load	0x20001080	0x2000	0001000010 ₂ →66 ₁₀	000000 ₂	miss
■ Load	0x20001008	0x2000	0001000000 ₂ →64 ₁₀	001000 ₂	hit



Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

	address	tag	index	offset	
Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
Store	0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss
Load	0x20001080	0x2000	0001000010 ₂ →66 ₁₀	000000 ₂	miss
Load	0x20001008	0x2000	0001000000 ₂ →64 ₁₀	001000 ₂	hit
Load	0x40811024	0x4081	0001000000 ₂ →64 ₁₀	100100 ₂	miss, write back 0x36801038

set 64에 replace된 후,
 2way victim target의 dirty bit가 1
 ⇒ write back으로 main memory로
 보낼 dirty bit는 0으로 바뀜

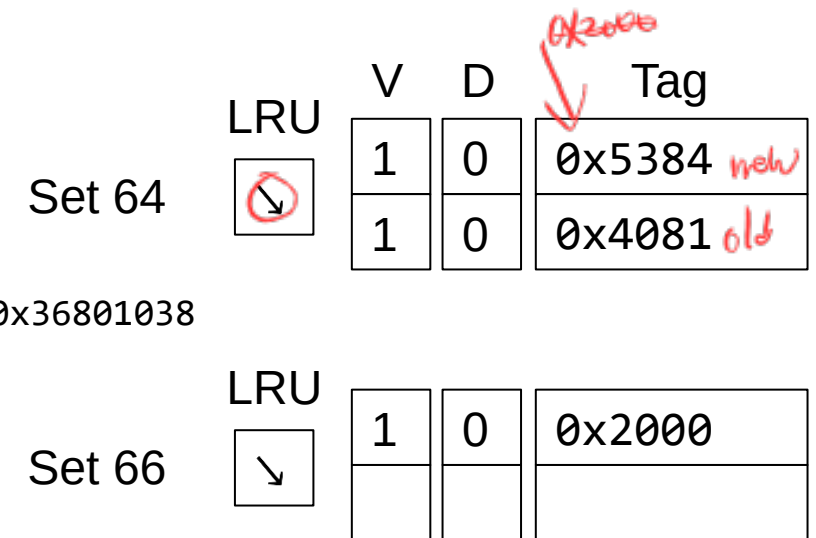
	LRU	V	D	Tag
Set 64	↗	1	0	0x2000 old
		1	0	0x4081 new
Set 66	↘	1	0	0x2000

Handwritten notes: 0x3680 → main memory

Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

		address	tag	index	offset	
▪	Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
▪	Store	0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss
▪	Load	0x20001080	0x2000	0001000010 ₂ →66 ₁₀	000000 ₂	miss
▪	Load	0x20001008	0x2000	0001000000 ₂ →64 ₁₀	001000 ₂	hit
▪	Load	0x40811024	0x4081	0001000000 ₂ →64 ₁₀	100100 ₂	miss, write back 0x36801038
▪	Load	0x53841010	0x5384	0001000000 ₂ →64 ₁₀	010000 ₂	miss



Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

	address	tag	index	offset	
■ Load	0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
■ Store	0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss
■ Load	0x20001080	0x2000	0001000010 ₂ →66 ₁₀	000000 ₂	miss
■ Load	0x20001008	0x2000	0001000000 ₂ →64 ₁₀	001000 ₂	hit
■ Load	0x40811024	0x4081	0001000000 ₂ →64 ₁₀	100100 ₂	miss, write back 0x36801038
■ Load	0x53841010	0x5384	0001000000 ₂ →64 ₁₀	010000 ₂	miss
■ Store	0x53841020	0x5384	0001000000 ₂ →64 ₁₀	100000 ₂	hit

Set 64

LRU

↘

V	D	Tag
1	1	0x5384
1	0	0x4081

Set 66

LRU

↘

1	0	0x2000

Cache Example

- 128KB data capacity, 64 bytes per block, 32bit address
- 2-way set associative, LRU replacement
- Store policy: write-back, write-allocate.

	address	tag	index	offset	
▪	Load 0x20001000	0x2000	0001000000 ₂ →64 ₁₀	000000 ₂	miss
▪	Store 0x36801038	0x3680	0001000000 ₂ →64 ₁₀	111000 ₂	miss
▪	Load 0x20001080	0x2000	0001000010 ₂ →66 ₁₀	000000 ₂	miss
▪	Load 0x20001008	0x2000	0001000000 ₂ →64 ₁₀	001000 ₂	hit
▪	Load 0x40811024	0x4081	0001000000 ₂ →64 ₁₀	100100 ₂	miss, write back 0x36801038
▪	Load 0x53841010	0x5384	0001000000 ₂ →64 ₁₀	010000 ₂	miss
▪	Store 0x53841020	0x5384	0001000000 ₂ →64 ₁₀	100000 ₂	hit
▪	Load 0x20001008	0x2000	0001000000 ₂ →64 ₁₀	001000 ₂	miss

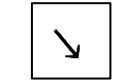
	V	D	Tag
Set 64	1	1	0x5384
Set 64	1	0	0x2000
Set 66	1	0	0x2000
Set 66			

LRU



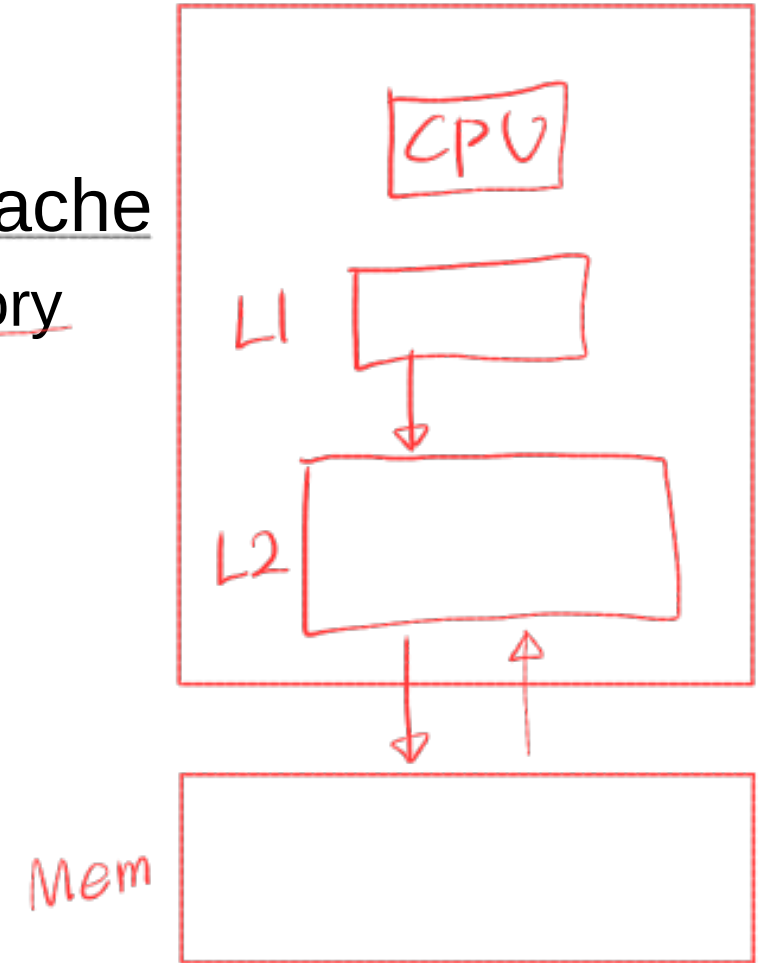
0x4081 ↑

LRU



Multi-level Caches

- Primary cache^{L1} attached to CPU
 - ❖ Small, but fast
- Level-2 cache handles misses from the L-1 cache
 - ❖ Larger, slower, but still faster than the main memory
L1 miss? → L2
- Main memory handles L-2 cache misses
 - L2 miss? → main memory*
- Some high-end systems include L-3 cache
 - ❖ Sometimes called the last-level cache or LLC.



Multi-level Cache Example

■ Given

- ❖ CPU base CPI = 1, clock rate = 4GHz
- ❖ Miss rate/instruction = 2%
- ❖ Main memory access time = 100ns

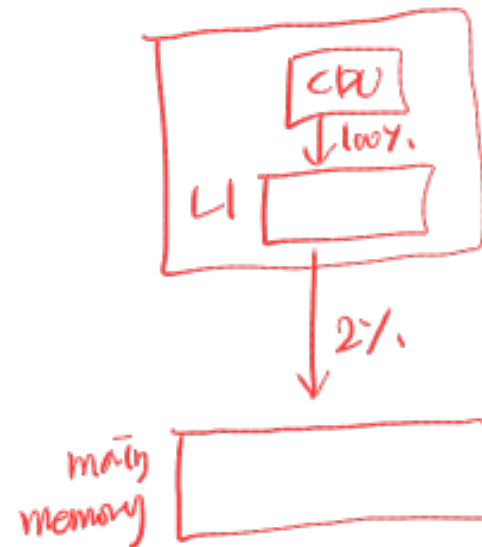
1ns = 4 clock cycles clock 0.25ns

400 cycles ← miss penalty

■ With just primary cache *LI만 앞다면*

- ❖ Miss penalty = $100\text{ns} / 0.25\text{ns} = 400$ cycles
- ❖ Effective CPI = $1 \times 100\% + 400 \times 2\% = 9$

1 + 8



Example (cont.)

■ Now add L-2 cache

- ❖ Access time = 5ns *20 cycles*
- ❖ **Global** miss rate to main memory = 0.5%
 - **Local** L2 cache miss rate?

2.5%의 0.5% → L2 miss

L2에 2.5% miss하는건

■ Primary miss with L-2 hit

- ❖ Penalty = $5\text{ns} / 0.25\text{ns} = 20$ cycles

■ Primary miss with L-2 miss

- ❖ **Extra** penalty = 400 cycles

■ $\text{CPI} = 1 \times 100\% + 20 \times 2\% + 400 \times 0.5\% = 3.4$

■ $\text{Performance ratio} = 9 / 3.4 = 2.6 \times \text{faster with L2}$

L1만 있는 것보다 L2도 있는 게 2.6배 더 빠름

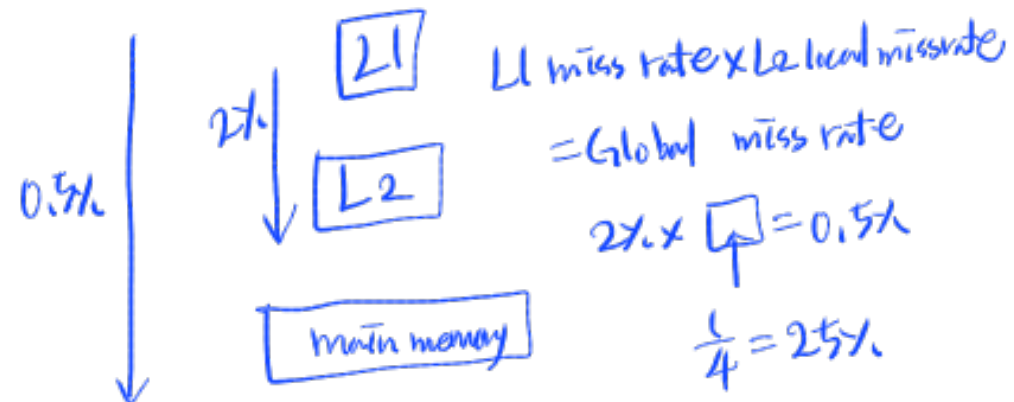
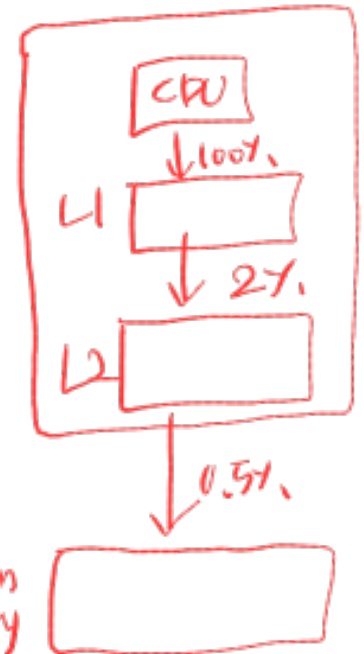
$= 1 \times 100\% + 2\% (20 \times 100\% + 400 \times 25\%)$ average delay

of L2 misses $\Rightarrow \frac{0.5\%}{2\%} = 25\%$

*왜 L2에서 miss되는 비율이 높아지는가,
frequently access data는
L1 cache에서 이미 hit될
반면에 L2는 다른 개체들이
infrequently access data를
완전 가능하게 더함*



local



Multi-level Cache Considerations

- Primary (L-1) cache

- ❖ Focus on minimal hit time

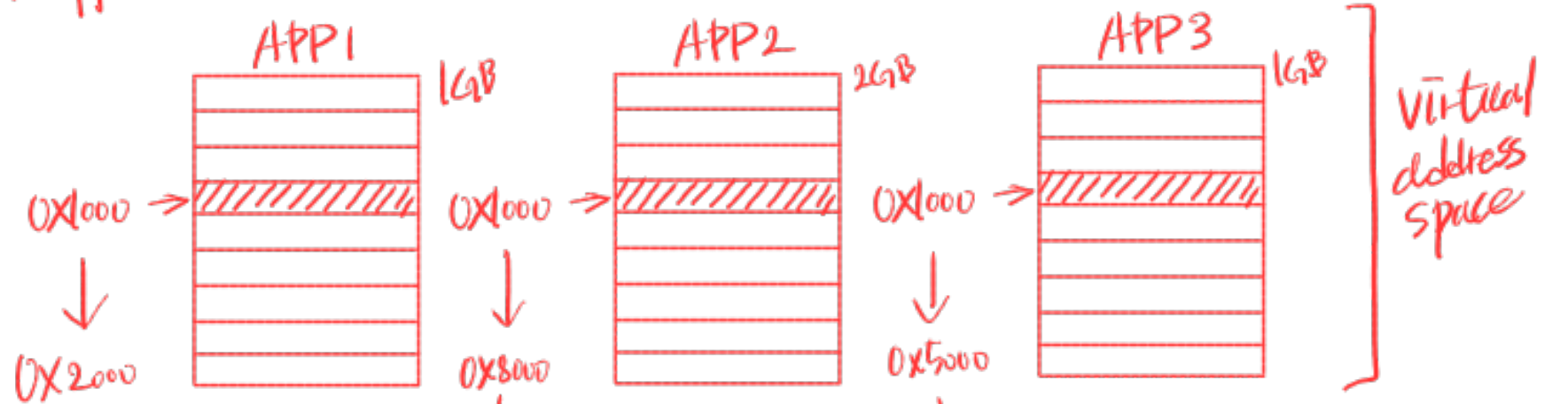
- L-2 cache

- ❖ Focus on low miss rate to avoid main memory access
- ❖ Hit time has less overall impact

→ L1의 hit rate이 높으면
L2의 hit time의 영향↓

가끔 L1, L2의 block size가
다른 경우도 있음

여러 application을 동시에 실행



compile 이후 address가 중복된 경우
다른 application이 같이 실행할 수 없음
→ 같은 address를 사용하게 될 수 없음

VIRTUAL MEMORY

다른 주소로 mapping될
→ collision 없이 여러 App을 실행

Hardware address



3GB

↓ main memory
App 1, 2, 3이 사용하는
memory를 감할 수 없음
→ virtual memory로 해결

physical address space

Why Virtual Memory?

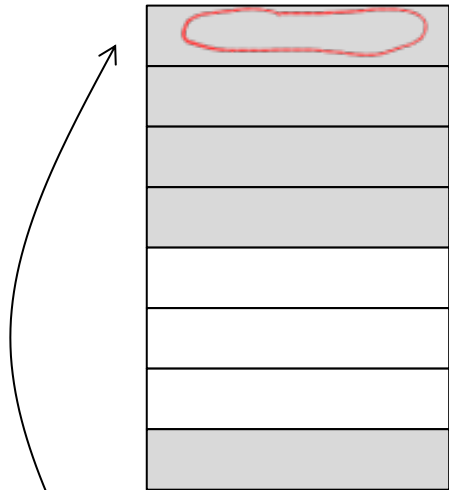
- Each program gets a private virtual address space
 - ❖ Multiple programs share main memory
 - ❖ Do not need to be programmed for a specific memory size
- If there isn't enough main memory, relocates some data to storage (disk).
 - ❖ Managed jointly by CPU hardware and the operating system (OS)

machine와 다른 main memory size,
address mapping을 application마다
memory size를 맞출 필요가

Virtual Memory

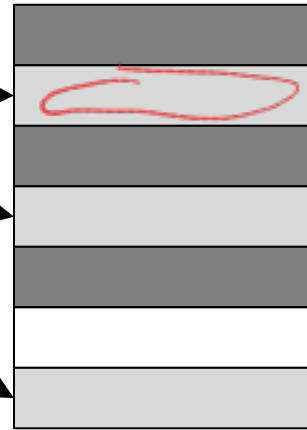
memory의 용량이 main memory보다 더 작을 경우
여러 applications을 동시에 실행 가능

Program 1
Virtual Address Space



“Page”
4KB

Main Memory
Physical Address Space

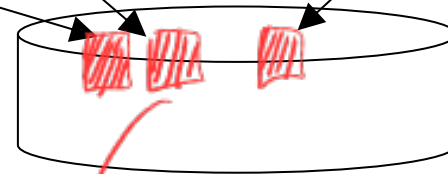


mapping management → OS

Program 2
Virtual Address Space



page가 너무 크면!!!
page의 수를 줄일수록
less efficiently



Disk Access

논리적으로 접근하는 것보다
물리적으로 접근하는 것

Virtual Address

User application

load
store

address

0x4124

← **Virtual address**

Translate

address

0x30124

← **Physical address**

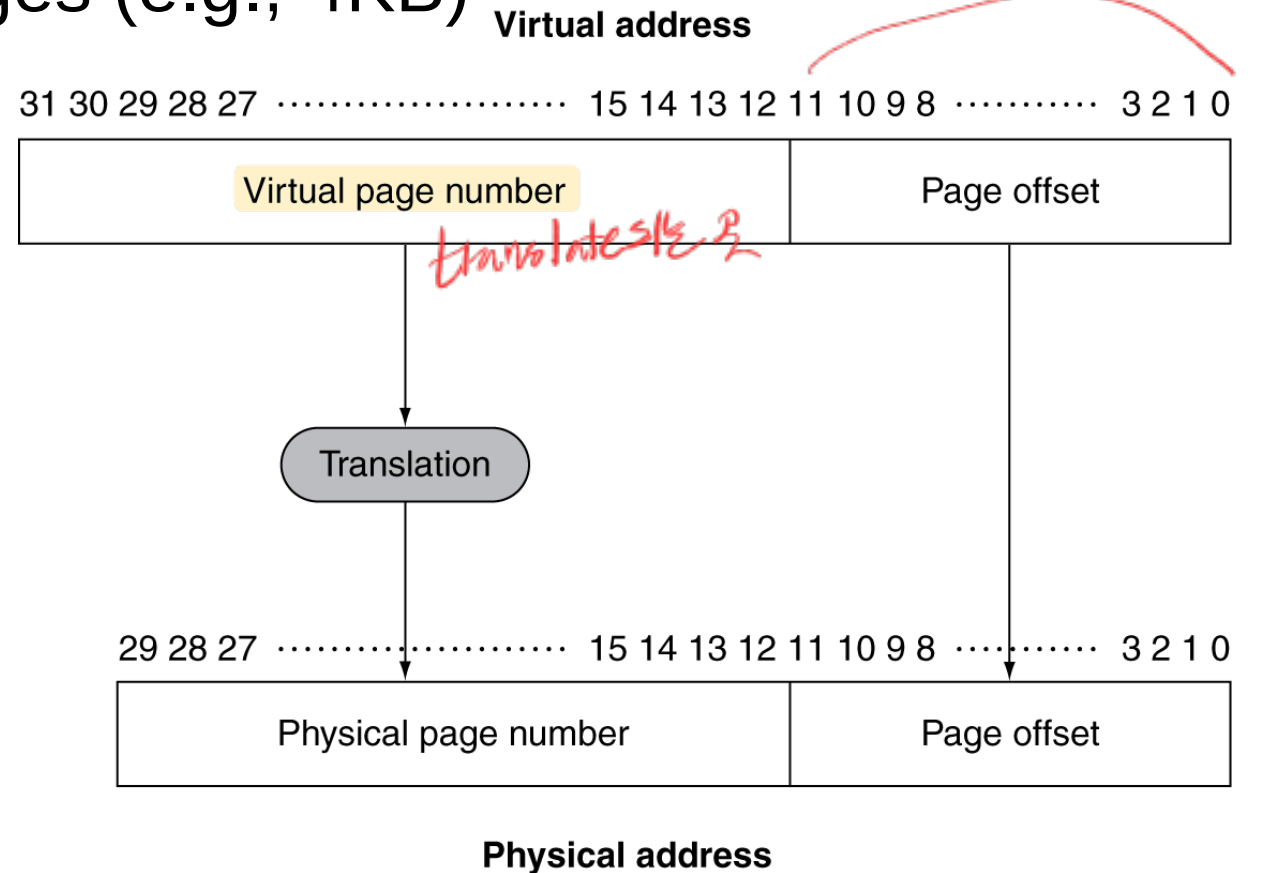
Memory (cache)

lower
bits 같고
page size 때문

동일한 access 방식

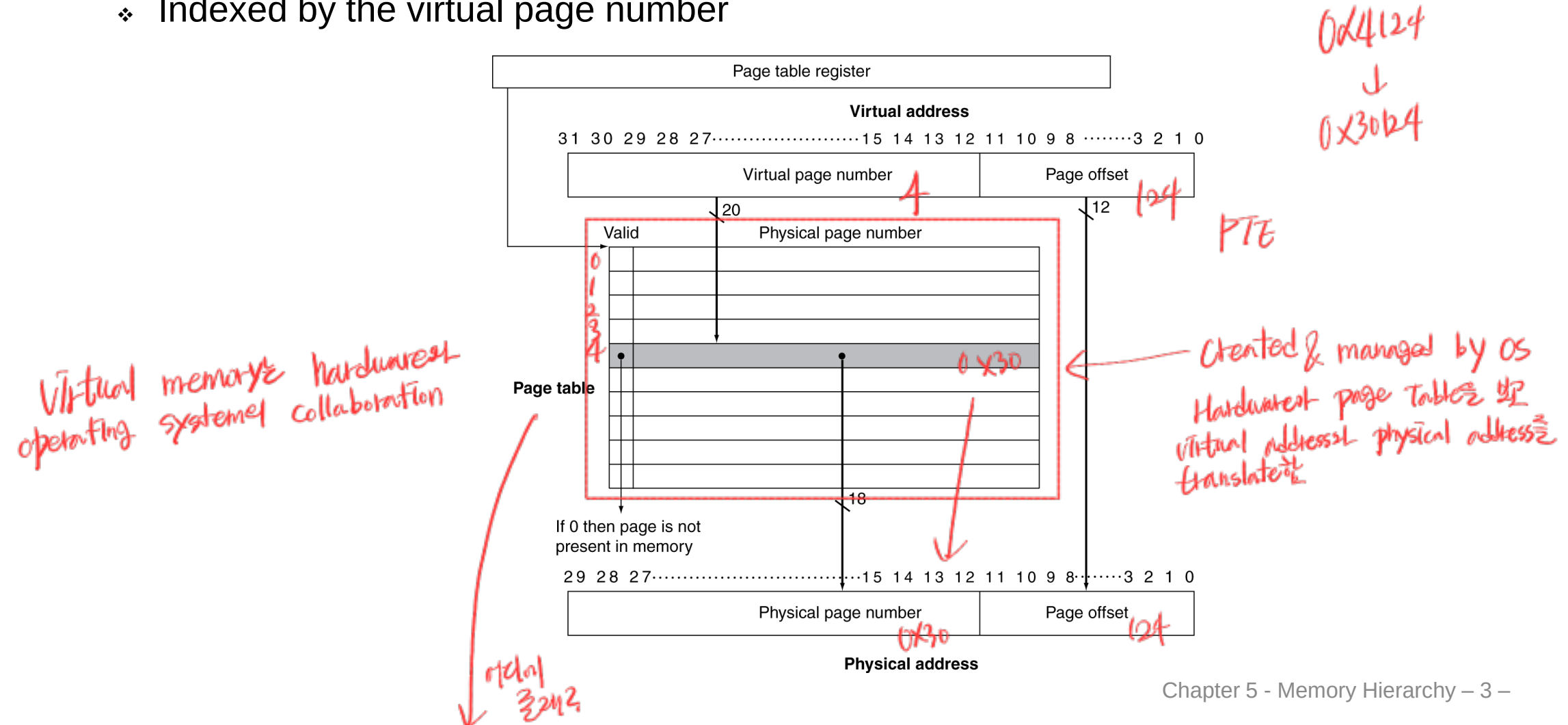
Address Translation

- Fixed-size pages (e.g., 4KB)



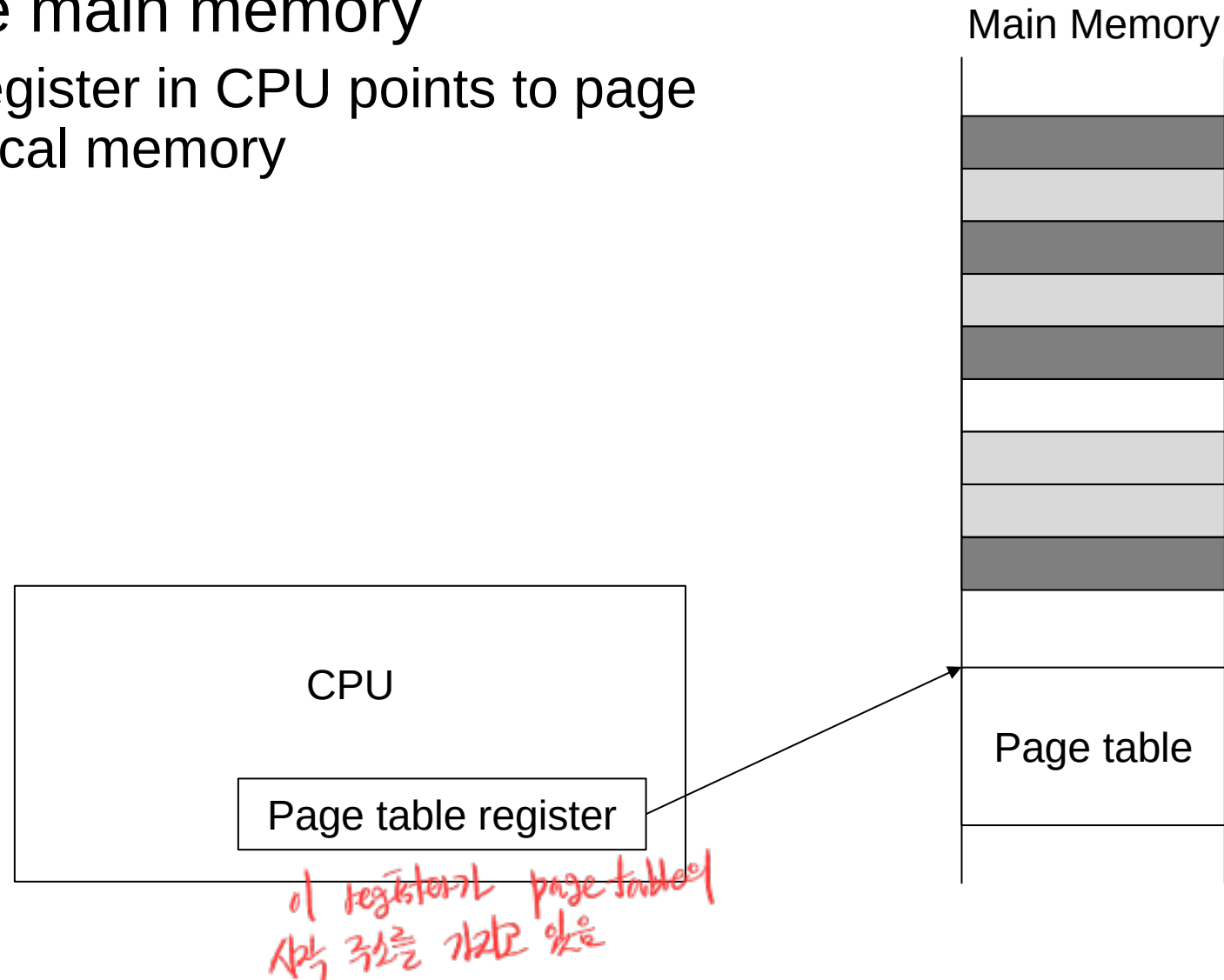
Translation Using a Page Table

- Stores page mapping information
 - ❖ Array of **page table entries**
 - ❖ Indexed by the virtual page number



Page Table in Memory

- Resides in the main memory
 - ❖ Page table register in CPU points to page table in physical memory



Page Table Entry (PTE)

■ If page is present in memory

- ❖ PTE stores the physical page number
- ❖ Plus, other status bits (valid, dirty, referenced, ...)

Dirty bit :
0이면 디스크로 보내지 않음
(이러야 할 때)
write back하면 1이 되면
replace

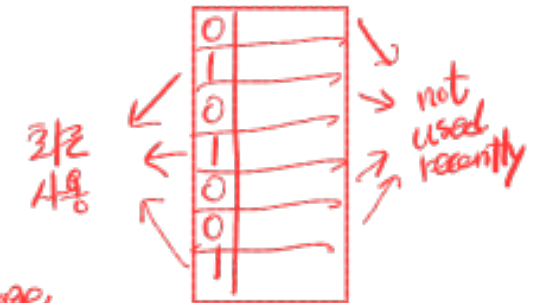
Referenced bit :
pseudo LRU
1이면 recently referenced
(used)
0을 victim으로

■ If page is not present (i.e., valid bit is 0)

- ❖ This means the page is in the disk or not mapped to memory.
- ❖ Accessing the page → **Page fault**

Valid bit 0이 되면
→ segmentation fault

→ unallocated



Page table

PTE for virtual page 0

PTE for virtual page 1

PTE for virtual page 2

⋮

V	D	R	Physical page number
V	D	R	Physical page number
V	D	R	Physical page number

store하면
D→1

store/load
R→1 4 bytes → for every 4KB page

status bit

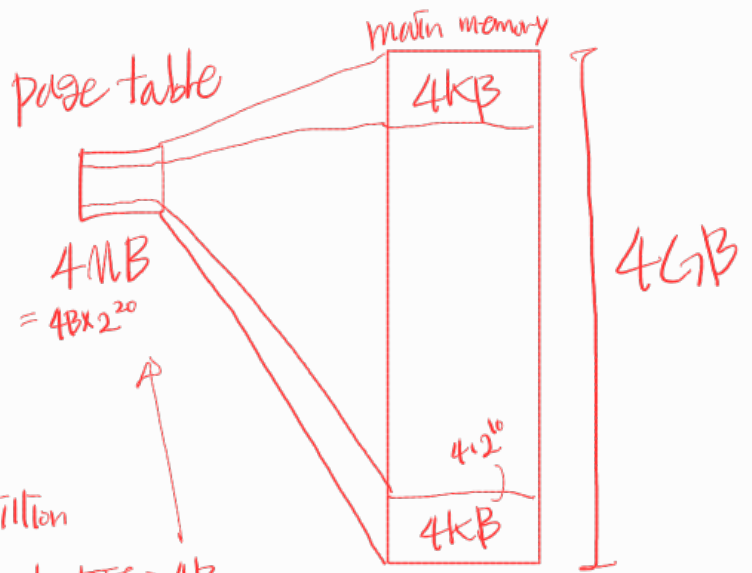
Valid bit = 1 → main
memory에 존재

0 — page is disk space region
unallocated

여기서 참조하면
disk에서 다시
main memory로 옮기게 됨

4KB physical page에서
4B의 PTE가 존재
entire address가 72bit
page table을 가리킴

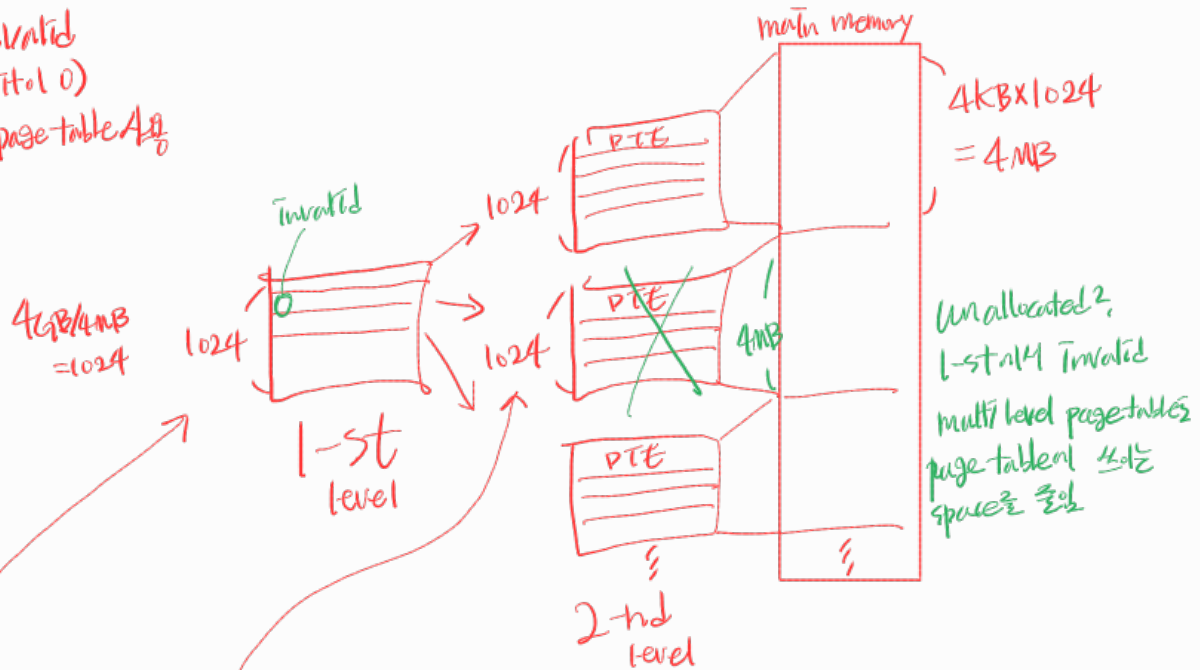
32bit address
 $2^{32} = 4GB$
 $4GB / 4KB = 2^{20}$
 $2^{20} = 1 \text{ million}$
 Each PTE = 4B



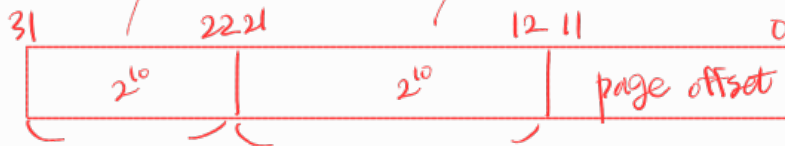
1 level
multi level

application은 보통 memory space를
작게 사용하러

대부분의 entry는 Invalid
(Valid bit = 0)
=> 불필요한 낭비, 지정한 page table 없음

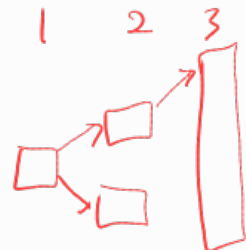


Unallocated, 1-st level Invalid
multi level page tables
page table의 쓰는
space를 줄임



multiple page table을
이용한 memory access를 위한
다단계 memory access 필요

level 2의 Pte
3번 access



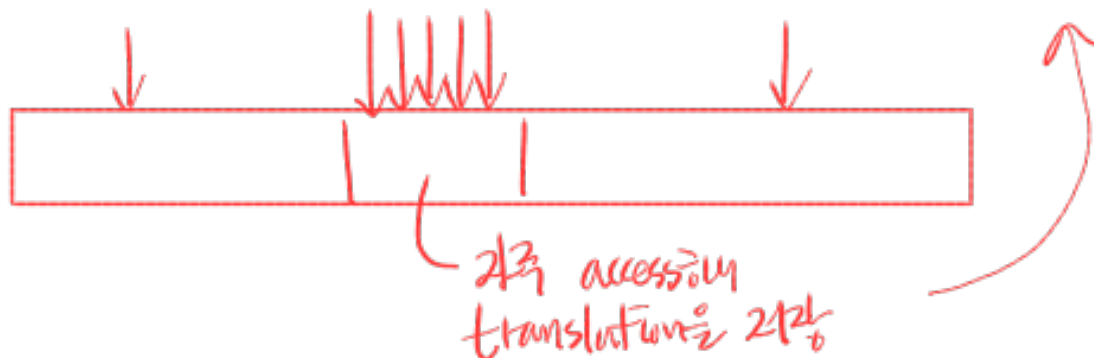
Page Fault Handler

- **Page fault** is handled by OS
 - ❖ Locate page on disk
 - ❖ Choose a page to replace
 - If dirty, write to disk first
 - ❖ Read page into memory and update page table

Valid bit → 1

Fast Translation Using a TLB

- Address translation would appear to require extra memory references
 - ❖ One to access the PTE
 - ❖ Then the actual memory access
- Usually, accesses to page table has good locality patterns
 - ❖ Use a fast cache of PTEs within the CPU
 - ❖ Called a Translation Look-aside Buffer (TLB)

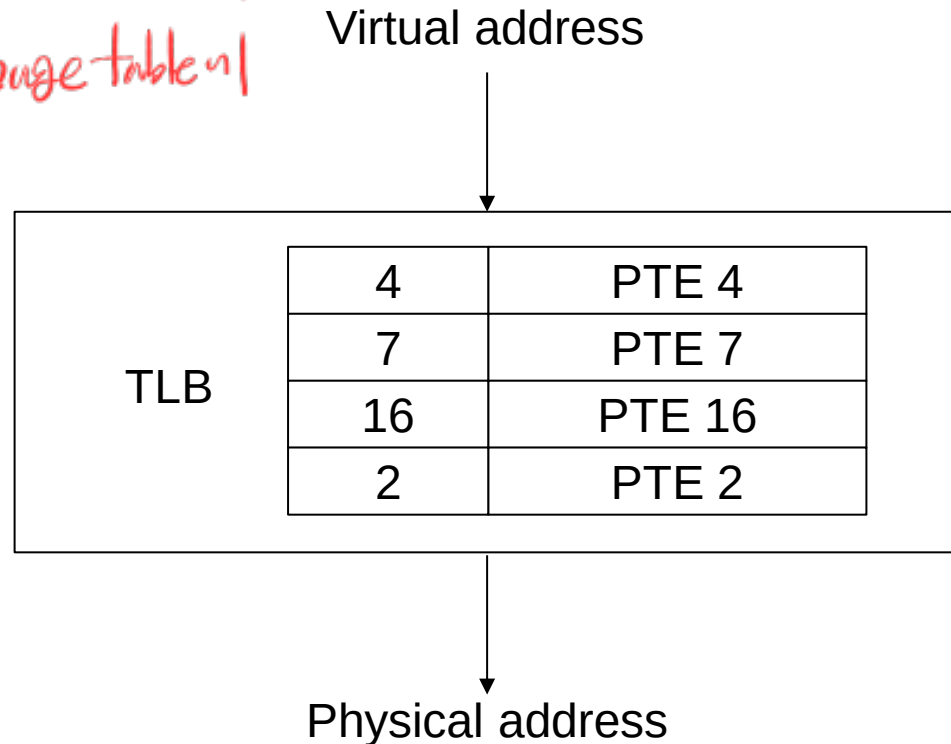


PTE를 가리키고
TLB를 확인

TLB

- Store frequently accessed PTEs in a faster memory
- Typical: 16–512 PTEs
- Fully associative, n-way set associative, ...

virtual → physical로의 translated 필요할 때 TLB부터 확인
hit → main memory의 page table에
가지 안가고 됨
miss → update TLB



Main Memory
Page table
PTE 0
PTE 1
PTE 2
PTE 3
PTE 4
PTE 5
⋮
PTE n

TLB Misses

- TLB miss: The page table entry (PTE) is not in TLB
 - ❖ This can be handled by the CPU (hardware TLB management) or by the OS (software TLB management)
 - x86: hardware TLB management
 - MIPS: software TLB management

RISC-V: SW/HW
- If PTE says...
 - ❖ The page is in memory
 - Load the PTE from memory to TLB and retry the memory access
 - ❖ The page is not in memory (page fault)
 - OS handles fetching the page and updating the page table

→ valid bit = 0

Memory Access Scenarios (Data load)

